AutoTabber

Audio to Beginner Guitar Tablature

Course Project: Milestone Submission

Team Members:

Daniel Ahn

Shauryaditya Singh

GitHub: To be uploaded

Demo

Python 3.12 | Streamlit | librosa | TensorFlow/Basic-pitch | SQLite

TABLE OF CONTENTS

1. Project Description....3

2. Project Requirements....4

2.1 Language and Python Version....4

2.2 GitHub Repository....4

2.3 Classes....4

2.4 Multiple .py Files....5

2.5 Graphical User Interface....5

2.6 Interactive GUI Elements....5

2.7 Advanced Python Module....6

2.8 Data I/O....7

2.9 Testing with pytest....7

2.10 Code Documentation....7

2.11 User Instructions....8

3. Milestone Submission....9

3.1 Basic Functionality....9

3.2 User Interactions....10

3.3 Files Included....10

3.4 How to Run....11

3.5 Known Limitations....11

1. Project Description

AutoTabber is a Python web application that takes an audio file and converts it into beginner-friendly guitar tablature. The idea came from the frustration of trying to learn songs on guitar without having access to accurate tabs. Most tabs online are user-submitted and often wrong, and commercial transcription software tends to be expensive and overly complex for someone just picking up the instrument. We wanted to build something that could listen to a song and produce a simple, playable tab that a beginner could actually follow.

The app accepts MP3, WAV, and M4A audio files. Once a file is uploaded, the user picks one of two pitch detection backends. The first is pYIN, a classical algorithm from librosa that tracks one note at a time. It works well on clean recordings with a single instrument. The second is Basic-pitch, a convolutional neural network developed by Spotify that can detect multiple notes playing at the same time. This means it can handle chords and full-band recordings, which is a big deal because most real-world music is polyphonic.

After pitch detection, the raw note data goes through several processing stages. A beat-grid quantizer snaps the notes to musically meaningful time slots (quarter notes, eighth notes, or sixteenth notes depending on user preference). A fretboard mapper then figures out where each note should be played on the guitar neck, prioritizing open strings and lower fret positions because those are easier for beginners. The final output is a standard six-line ASCII guitar tab that the user can read, download as a text file, or access later through a built-in tab history backed by a SQLite database.

The entire interface runs through Streamlit, which gives us a clean browser-based GUI without needing to deal with frontend frameworks. All the audio processing, pitch detection, quantization, and fretboard mapping happens on the backend in Python. The app is also deployed on Streamlit Cloud so that anyone with a browser can try it out without installing anything locally.

2. Project Requirements

Below is a walkthrough of how AutoTabber satisfies each requirement from the project specification.

2.1 Language and Python Version

The entire application is written in Python. Our local development environment runs Python 3.12.10, and the Streamlit Cloud deployment uses Python 3.12.13. All code uses type hints and features available in Python 3.12.

2.2 GitHub Repository

Our code will be hosted on GitHub at github.com/shauryadata/Auto_Tabber on the main branch. Both team members will be making substantial commits to the repository throughout the development process. Commits include building out classes, adding pitch detection logic, writing tests, implementing the GUI, and deploying the application.

2.3 Classes

AutoTabber has seven distinct classes, each handling a specific part of the pipeline. Every class is stored in its own .py file inside the beginner_tab package and imported into app.py as a module.

Class	File	Purpose
AudioLoader	audio_loader.py	Loads MP3/WAV/M4A files, converts to WAV via pydub, returns a normalized mono NumPy array at 22,050 Hz
PitchTracker	pitch_tracker.py	Monophonic pitch detection using librosa's pYIN algorithm with optional HPSS preprocessing
BasicPitchTracker	pitch_tracker.py	Polyphonic pitch detection using Spotify's Basic-pitch CNN with ONNX backend
TabSimplifier	tab_simplifier.py	Quantizes raw note events onto a beat grid and collapses sustained notes

FretboardMapper	fretboard_mapper.py	Maps MIDI notes to guitar string/fret positions with beginner-friendly preferences
TabRenderer	tab_renderer.py	Renders mapped positions as a standard six-line ASCII guitar tab
TabStorage	tab_storage.py	CRUD operations for saving, listing, viewing, and deleting tabs in a SQLite database

2.4 Multiple .py Files

The project is organized into a main application file and a package of modules:

- app.py: Streamlit frontend that ties everything together
- beginner_tab/__init__.py: Package initializer that re-exports all classes
- beginner_tab/audio_loader.py: AudioLoader class
- beginner_tab/pitch_tracker.py: PitchTracker and BasicPitchTracker classes
- beginner_tab/tab_simplifier.py: TabSimplifier class
- beginner_tab/fretboard_mapper.py: FretboardMapper class
- beginner_tab/tab_renderer.py: TabRenderer class
- beginner_tab/tab_storage.py: TabStorage class
- tests/: Four test modules covering the pipeline end to end

Each class is imported from its module into app.py through the package's __init__.py, keeping the main application file clean and readable.

2.5 Graphical User Interface

The GUI is built with Streamlit, which renders as a full browser-based web application. Users interact entirely through the browser. There is no terminal interaction or text-based interface required. The layout includes a sidebar for settings and a main content area for results.

2.6 Interactive GUI Elements

The application has well over five interactive elements. Here is a breakdown of the key ones:

#	Element	What It Does
1	File Uploader	Lets the user drag and drop or browse for an MP3, WAV, or M4A audio file
2	Detector Radio Button	Selects between Basic-pitch (polyphonic AI) and pYIN (monophonic) pitch detection
3	Max Fret Slider	Controls the highest fret allowed in the output tab (range 3 to 12)
4	Note Grid Selectbox	Sets the quantization resolution: quarter notes, eighth notes, or sixteenth notes
5	One-String Checkbox	Restricts output to the high e string only for absolute beginners
6	Columns Per Line Slider	Controls how many note columns appear per line in the tab output
7	Generate Tab Button	Runs the full pipeline: load, detect, quantize, map, render
8	Download Button	Downloads the generated tab as a .txt file with metadata header
9	Advanced Tuning Sliders	Onset threshold, frame threshold, min note length (Basic-pitch) or confidence and HPSS toggle (pYIN)
10	Tab History Controls	View, download, or delete previously generated tabs from the SQLite history

2.7 Advanced Python Module

Our primary advanced module is TensorFlow, which powers the Basic-pitch neural network for polyphonic pitch detection. Basic-pitch is a convolutional neural network trained by Spotify on a large dataset of real music recordings. It takes raw audio as input and outputs timestamped MIDI note events, including chords. Internally, the model runs through the ONNX Runtime backend for faster inference, but the model itself was trained and exported using TensorFlow.

We also make heavy use of librosa for the pYIN pitch detection algorithm, Harmonic-Percussive Source Separation (HPSS), beat tracking, and frequency-to-MIDI conversion. While librosa is

technically a DSP library rather than a machine learning framework, its pYIN implementation involves probabilistic modeling and is a core part of our processing pipeline.

SQLite (via Python's built-in sqlite3 module) handles all persistent data storage. The TabStorage class manages a local database where every generated tab is saved automatically with metadata like the detector used, tempo, max fret setting, and a UTC timestamp.

2.8 Data I/O

AutoTabber performs meaningful data I/O in two directions:

- Input: The AudioLoader class reads audio files from disk. For non-WAV formats (MP3, M4A), it uses pydub with ffmpeg to decode and convert the audio into a temporary WAV file. librosa then loads this WAV as a NumPy array at 22,050 Hz mono.
- Output/Storage: Generated tabs are saved to a SQLite database file (tabs.db) through the TabStorage class. The app performs full CRUD operations: creating new records on tab generation, reading them for the history view, and deleting them at the user's request. Users can also download tabs as .txt files.

2.9 Testing with pytest

We have four test files in the tests/ directory covering the core pipeline components:

- test_fretboard_mapper.py: Tests position lookup for known MIDI notes, max-fret enforcement, one-string mode behavior, and the movement-cost ranking logic
- test_tab_renderer.py: Tests MIDI-to-note-name conversion, tab structure (six strings, correct labels), line wrapping, and column width calculations
- test_tab_simplifier.py: Tests slot duration math, quantization accuracy, range simplification, and sustained-note collapse
- test_chord_pipeline.py: End-to-end tests for the polyphonic pipeline including chord mapping, chord rendering, and TabStorage CRUD operations

The test suite has well over five meaningful test cases across these files. Tests use synthetic note data and in-memory SQLite databases so they run fast without requiring audio files or model downloads.

2.10 Code Documentation

Every .py file has a header comment block that includes the authors, date, and a description of what the file does. All classes and public methods have docstrings explaining their purpose, arguments, and return values. Key variables and logic blocks are annotated with inline comments. For example, the pitch_tracker.py file explains what each pYIN return value represents, why we apply HPSS before detection, and what each post-processing filter in BasicPitchTracker is filtering out.

2.11 User Instructions

The app includes a collapsible "How to Use AutoTabber" section at the top of the main page. It walks the user through five steps: upload a file, pick a detector, adjust the max fret, click Generate Tab, and download the result. Every sidebar control has a help tooltip that appears on hover. There is also a "Known Limitations and Tips" section at the bottom that explains the differences between the two detectors and gives guidance on tuning parameters.

3. Milestone Submission

This section covers the current state of AutoTabber for the milestone submission and demonstrates that the core functionality is working.

3.1 Basic Functionality

The milestone version of AutoTabber delivers the following working features:

- **Audio Loading:** Users can upload MP3, WAV, or M4A files up to 50 MB. The AudioLoader class handles format conversion and normalization. The app validates file size, checks for silence, and warns about very short or very long recordings.
- **Pitch Detection:** Both detection backends are functional. pYIN handles single-note melodies with optional HPSS preprocessing. Basic-pitch handles chords and polyphonic audio through its ONNX-backed neural network. Both apply guitar-range frequency bandpass (82 Hz to 1175 Hz) and MIDI range filtering (MIDI 28 to 74).
- **Beat Quantization:** Raw note events are snapped to a musical grid based on the detected tempo. The user can select quarter, eighth, or sixteenth note resolution. Sustained notes held across adjacent beats are collapsed into single events.
- **Fretboard Mapping:** MIDI notes are placed on the guitar fretboard using a beginner-friendly strategy. Open strings always get priority. The mapper minimizes hand movement between consecutive notes. Notes above the user's max-fret setting are skipped, and the user is told how many were dropped.
- **Tab Rendering:** The output is a standard six-line ASCII guitar tab with string labels (e, B, G, D, A, E), fret numbers, and a note-name legend. Lines wrap automatically based on the user's columns-per-line setting.
- **Data Persistence:** Every generated tab is automatically saved to a SQLite database with metadata. The Tab History section lets users browse, view, download, or delete past results.
- **Deployment:** The app is live on Streamlit Cloud at autotabber.streamlit.app. The cloud version runs the pYIN detector because Basic-pitch's TensorFlow dependency is not compatible with the cloud environment's Python version. The local version supports both detectors.

3.2 User Interactions

A typical user session looks like this: the user opens the app, uploads a guitar recording, selects Basic-pitch as the detector, moves the max fret slider to 5, and clicks Generate Tab. The app shows a progress indicator with four stages (loading audio, running the detector, quantizing to beat grid, mapping to fretboard). When it finishes, the tab appears on screen with a success message showing the number of events and estimated tempo. The user can then download the tab as a .txt file or find it later in the Tab History section.

If the user wants to experiment, they can change the detector, adjust advanced parameters like the onset threshold, try a different max fret setting, and regenerate. Each run produces a new entry in the history, so the user can compare results side by side.

3.3 Files Included

The following files are included in the repository and are required to execute the project:

File	Description
app.py	Main application file, Streamlit frontend
beginner_tab/__init__.py	Package initializer with availability probe
beginner_tab/audio_loader.py	AudioLoader class for file loading
beginner_tab/pitch_tracker.py	PitchTracker and BasicPitchTracker classes
beginner_tab/tab_simplifier.py	TabSimplifier class for beat quantization
beginner_tab/fretboard_mapper.py	FretboardMapper class for fret placement
beginner_tab/tab_renderer.py	TabRenderer class for ASCII output
beginner_tab/tab_storage.py	TabStorage class for SQLite persistence
tests/test_fretboard_mapper.py	Unit tests for fretboard mapping
tests/test_tab_renderer.py	Unit tests for tab rendering
tests/test_tab_simplifier.py	Unit tests for beat quantization
tests/test_chord_pipeline.py	End-to-end tests for chord pipeline
requirements.txt	Cloud deployment dependencies

requirements-local.txt	Full local development dependencies
packages.txt	System packages for Streamlit Cloud (ffmpeg)

3.4 How to Run

To run AutoTabber locally with both detectors available:

1. Clone the repository: git clone https://github.com/shauryadata/Auto_Tabber.git
2. Create and activate a virtual environment: python3 -m venv venv && source venv/bin/activate
3. Install dependencies: pip install -r requirements-local.txt
4. Make sure ffmpeg is installed on your system
5. Launch the app: streamlit run app.py

The app will open in a browser. Both the pYIN and Basic-pitch detectors will be available in the sidebar.

To run the test suite: pytest tests/ -v

3.5 Known Limitations

- The Basic-pitch detector is only available when running locally with Python 3.12 or earlier, because it depends on TensorFlow which does not have pre-built packages for Python 3.13 and above.
- The generated tabs represent note sequence only. Rhythm durations (how long each note is held) are not encoded in the ASCII output.
- Pitch detection accuracy depends heavily on the quality and complexity of the input audio. Clean single-instrument recordings produce the best results.
- The fretboard mapper optimizes for hand position comfort, not for musical phrasing. Some fingerings may differ from how a skilled guitarist would play the passage.

We plan to continue refining the detection accuracy and exploring options like stem separation for the final submission.